



# MediTrack

## Full-Stack Engineering Walkthrough & Interview Guide

An event-driven Patient & Health-Records platform built on .NET 9 Clean Architecture, PostgreSQL, Redis, Kafka, AI, and Next.js 16.

.NET 9

Clean Architecture

PostgreSQL

Redis

Kafka

SignalR

OpenTelemetry

Next.js 16

Material UI

React Query

Prepared as a complete talking-track covering backend, frontend, services, data, messaging, real-time, security, and AI.

# Contents

---

1. **The 60-second pitch** — what MediTrack is and who uses it
2. **System architecture** — the big picture & request lifecycle
3. **Technology stack** — every technology and why it's there
4. **Backend — Clean Architecture** — the four .NET projects + Workers
5. **Domain & data layer** — entities, EF Core, stored procedures
6. **Application services** — the business logic, service by service
7. **Event-driven architecture** — Kafka topics, producers, consumers, DLQ
8. **Real-time layer** — SignalR + Redis backplane
9. **Security & authentication** — JWT, RBAC, refresh rotation, audit trail
10. **AI subsystem** — symptom analysis, RAG chat, record summarization
11. **Cross-cutting concerns** — caching, resilience, health, observability, rate limiting
12. **Frontend architecture** — Next.js, state, data, realtime, theming
13. **End-to-end feature walkthroughs** — tying it all together
14. **Interview Q&A** — likely questions and strong answers

## 1 · The 60-Second Pitch

MediTrack is a clinic management & patient health-records platform. It has two faces sharing one backend: a **clinic app** for staff (doctors & receptionists) and a self-service **patient portal**.

### What staff can do

- Search patients, view profiles, history, prescriptions & records
- Book appointments and drive a status workflow (Scheduled → Checked-in → Completed / Cancelled / No-show)
- Record diagnoses and issue prescriptions from an appointment
- Upload medical documents that get **AI-summarized asynchronously**
- Triage prescription refill requests (approve / deny with notes)
- Provision patient portal accounts & message patients securely
- Use an AI assistant: symptom analyzer + patient-history chat
- See a dashboard: 5 KPI tiles + 7-day trend chart + today's schedule

### What patients can do (portal)

- View their own profile, appointments, prescriptions & AI-summarized records
- Self-book appointments
- Request prescription refills
- Message the clinic (secure thread)

Every portal request is scoped to the logged-in patient — staff-only endpoints return **403**, and patients can never read another patient's data.

**Framing line for an interviewer:** "MediTrack is a production-shaped MVP that demonstrates Clean Architecture in .NET, an event-driven backbone over Kafka, real-time push via SignalR, a proper security model with JWT + RBAC + an immutable audit trail, and a modern Next.js 16 front end — all wired to run locally with Docker in one command."

**Honest scope note (say this up front):** the MVP uses synthetic/seed data only — no real PHI. HIPAA/BAA compliance is intentionally deferred; the design is built *so that* it could be added (audit trail, RBAC, storage abstraction) without re-architecting.

## 2 · System Architecture

Five deployable/logical pieces communicate over HTTP, WebSockets, and Kafka:



### Why this shape?

- **API stays fast & synchronous** for reads/writes; anything slow or best-effort (AI summarization, audit persistence, notifications) is **offloaded to Kafka** so the HTTP request returns immediately.
- **Workers are a separate process** — they scale independently and a crash in AI processing never takes down the API.
- **Redis plays three roles:** cache-aside store, refresh-token store, and SignalR backplane (so real-time push still works when the API is horizontally scaled).
- **The API also hosts one consumer** ( `RecordSummarizedConsumer` ) — it listens for "summary ready" events and pushes them to the browser over SignalR. This is the bridge between the async world and the live UI.

### Request lifecycle (a typical authenticated read)

1. Browser sends request with `Authorization: Bearer <JWT>`.
2. `ExceptionHandlerMiddleware` wraps the pipeline (maps `AppException` → correct HTTP status).
3. JWT bearer auth validates issuer/audience/lifetime/signature; claims populate `ClaimsPrincipal`.
4. `[Authorize(Roles=...)]` enforces RBAC; rate limiter guards AI routes.
5. Controller calls a scoped **Application service** (interface, not implementation).
6. Service runs business logic → EF Core / stored procedure / Redis cache; may publish a Kafka event and write an audit event.
7. Typed DTO returns up the stack; the `ValidationFilter` /middleware shape the response.

### 3 · Technology Stack & Rationale

#### Backend

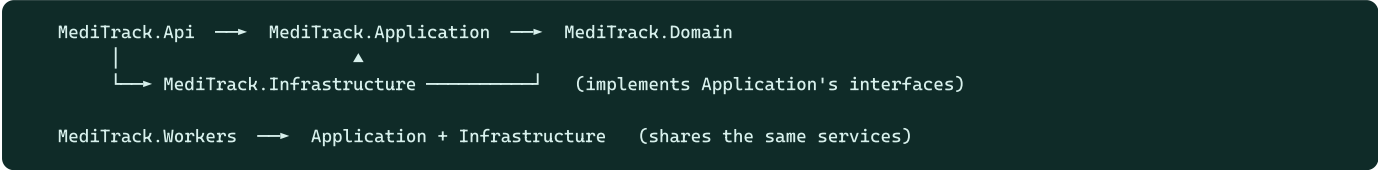
| Technology                 | Role                       | Why it was chosen                                                                  |
|----------------------------|----------------------------|------------------------------------------------------------------------------------|
| .NET 9 / ASP.NET Core      | API + Workers host         | Modern, fast, first-class DI, hosted services, minimal hosting model               |
| Clean Architecture         | Project structure          | Business logic isolated from frameworks & providers; testable; swappable adapters  |
| PostgreSQL 16              | Primary store              | Relational integrity, window functions, SQL stored functions for heavy reads       |
| EF Core 9 (Npgsql)         | ORM + migrations           | Productive writes, LINQ projections, code-first migrations, auto-seeding           |
| Redis 7                    | Cache / tokens / backplane | Sub-ms cache-aside, single-use refresh tokens, SignalR fan-out across replicas     |
| Apache Kafka               | Event bus                  | Durable, partitioned, at-least-once delivery; decouples slow work from the request |
| SignalR                    | Real-time push             | WebSocket abstraction with groups + Redis backplane for "summary ready" pushes     |
| FluentValidation           | Input validation           | Declarative rules, auto-registered, runs in a global filter                        |
| Polly (resilience handler) | AI HTTP resilience         | Retries, circuit breaker, timeout around the LLM call                              |
| OpenTelemetry              | Tracing + metrics          | Vendor-neutral observability; OTLP export or console in dev                        |
| BCrypt / JWT               | Security                   | Salted password hashing; stateless HMAC-signed access tokens                       |
| UglyToad.PdfPig            | PDF text extraction        | Pull text out of uploaded PDFs to feed the AI summarizer                           |
| xUnit                      | Tests                      | Unit tests over services & validators                                              |

#### Frontend

| Technology                    | Role              | Why                                                                 |
|-------------------------------|-------------------|---------------------------------------------------------------------|
| Next.js 16 (App Router)       | React framework   | File-based routing, route groups, layouts, server/client components |
| React 19 + TypeScript         | UI + type safety  | Modern React, typed API contracts                                   |
| Material UI v9 + MUI X Charts | Component system  | Accessible components, themable design system, dashboard charts     |
| Tailwind v4                   | Layout utilities  | Quick grid/spacing alongside MUI <code>sx</code>                    |
| TanStack React Query          | Server state      | Caching, invalidation, polling, mutations — the whole data layer    |
| Zustand                       | Client/auth state | Tiny persisted store for tokens/role; SSR-safe                      |
| Axios                         | HTTP client       | Interceptors for token attach + single-flight refresh               |
| @microsoft/signalr            | Realtime client   | Subscribes per-patient; invalidates queries on push                 |

## 4 · Backend — Clean Architecture

The solution is five projects. Dependencies point **inward**: outer layers depend on inner ones, never the reverse. The Application layer talks only to *interfaces*; Infrastructure provides the concrete adapters.



| Project        | Responsibility                                           | Key contents                                                                                                                                                                   |
|----------------|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Domain         | Enterprise entities & enums — pure C#, zero dependencies | Patient, Doctor, Appointment, Prescription, MedicalRecord, RefillRequest, Message, User, AuditLog; enums (roles, statuses); <b>AuditableEntity</b> base                        |
| Application    | Business logic + contracts. Depends only on abstractions | Service interfaces + implementations, DTOs, FluentValidation validators, Kafka event contracts, stored-proc helpers ( <b>SqlExecutor</b> ), <b>AppException</b>                |
| Infrastructure | Concrete adapters for everything external                | EF Core DbContext + configs + 5 stored procs, Redis cache & token store, Kafka producer + consumer base, AI providers (OpenAI/stub), BCrypt + JWT, file storage, PDF extractor |
| Api            | HTTP surface + composition root                          | Controllers, JWT/RBAC, SignalR hub + realtime consumer, rate limiting, health checks, OpenTelemetry, Swagger, middleware                                                       |
| Workers        | Background event processors (separate process)           | Kafka consumers: AI summarizer, notifications, audit persistence                                                                                                               |

**The payoff of this structure (a strong interview point):** the AI provider is behind **IAIService** , so the system ships with a deterministic **stub** that needs no API key and swaps to real OpenAI via one config value. Same story for storage ( **IFileStorage** = local disk now, S3/Blob later) and messaging. Business logic never changes when an adapter does.

### Dependency injection lifetimes (worth knowing)

- **Scoped** (per-request): all Application services, **DbContext** , **ICurrentUser** , cache service, audit logger, refresh-token store.
- **Singleton**: **IPasswordHasher** (BCrypt), **ITokenService** (JWT), **IEventPublisher** (one Kafka producer), **IFileStorage** , the stub AI service.
- **Typed HttpClient + Polly**: **OpenAIService** when the real provider is enabled.

## 5 · Domain & Data Layer

### Core entities

| Entity        | Notable fields / relationships                                                                                                       |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------|
| User          | Email (unique), PasswordHash, FullName, Role (Doctor/Receptionist/Patient), IsActive; optional 1:1 Doctor                            |
| Doctor        | FullName, Specialty, required link to User; has Appointments                                                                         |
| Patient       | FullName, DOB, BloodType, contacts, MedicalHistory; <b>optional</b> <code>UserId</code> — null until a portal account is provisioned |
| Appointment   | Patient + Doctor, ScheduledAt, Status enum, Reason, Diagnosis; has Prescriptions                                                     |
| Prescription  | Belongs to an Appointment; Medication/Dosage/Frequency; optional ExpiryDate (null = never expires)                                   |
| MedicalRecord | Patient, FileName, BlobKey (storage pointer), RecordType, <b>AI</b> Summary, <b>SummaryStatus</b> (Pending→Processing→Ready/Failed)  |
| RefillRequest | Patient + Prescription, Status (Requested/Approved/Denied), notes, resolver + timestamp                                              |
| Message       | Patient-keyed secure thread; SenderName, <code>FromPatient</code> flag, Body, ReadAt                                                 |
| AuditLog      | <b>Immutable</b> — never updated/deleted; ActorUserId, Action, EntityType, EntityId, Details, Timestamp                              |

### EF Core details worth mentioning

- **Centralized timestamps:** `DbContext.SaveChangesAsync` is overridden to stamp `CreatedAt` / `UpdatedAt` on any `AuditableEntity` — callers never set them.
- **Configuration by convention:** `ApplyConfigurationsFromAssembly` auto-discovers all `IEntityTypeConfiguration` classes; tables map to snake\_case.
- **Deliberate delete behaviors:** deleting a Doctor with appointments is `Restrict` ed; deleting a User `SetNull` s the patient link (the patient record survives); most children cascade.
- **Targeted indexes:** unique on User.Email & UserId links; composite `(DoctorId, ScheduledAt)` for schedules and `(PatientId, CreatedAt)` for message threads.
- **Auto-migrate + seed on startup** (`DbSeeder`): applies migrations, then idempotently seeds a doctor, receptionist, three patients, a sample appointment/prescription, and a demo patient-portal login — so a fresh clone is usable immediately.

### Stored procedures — reads pushed into PostgreSQL

Five `LANGUAGE sql STABLE` functions handle the heavy/aggregate reads. They're invoked through a small `SqlExecutor` ADO.NET helper so the raw-SQL provider never leaks into the Application layer. Why: single round-trips, window-function pagination, and DB-side aggregation instead of pulling rows into memory.

| Function                                                           | Purpose                                                                                                                                               |
|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>meditrack_dashboard_stats()</code>                           | All 5 KPIs (patients, today's appts, pending, active prescriptions, new-this-month) in one row via correlated subqueries                              |
| <code>meditrack_search_patients(term, blood, limit, offset)</code> | Paginated patient search with <code>ILIKE</code> + blood-type filter; total count via <code>COUNT(*) OVER()</code> window (page + total in one query) |
| <code>meditrack_today_appointments(doctor?)</code>                 | Today's appointments, optional doctor filter, joined to patient + doctor names                                                                        |
| <code>meditrack_patient_full_history(patient)</code>               | Full appointment history for a patient, newest first                                                                                                  |
| <code>meditrack_active_prescriptions(patient)</code>               | Non-expired prescriptions (ExpiryDate null or ≥ today)                                                                                                |

The search function was **versioned across two migrations** — the blood-type filter was added by dropping/recreating the function in a later migration, a nice example of evolving DB code under source control.

## 6 · Application Services (Business Logic)

Each service is an interface + implementation registered scoped. They orchestrate EF Core, stored procedures, caching, event publishing, and audit logging.

### AuthService

Login + refresh-token rotation. Verifies BCrypt password against an **active** user; on success issues a JWT access token + a random refresh token stored in Redis. Refresh is **single-use** (consumed & rotated). Failures return a deliberately generic "Invalid email or password" (no user enumeration).

### PatientService

CRUD + search. Search delegates to the stored procedure with clamped paging. Every read of a patient record **writes an audit event** ( `PatientRecordAccessed` ) — PHI-access tracking.

### AppointmentService

Validates patient & doctor exist, creates the appointment (status Scheduled), **publishes** `appointment.booked` to Kafka, audits. Also drives diagnosis and status transitions; history/today via stored procs.

### PrescriptionService

Creates a prescription from an appointment (deriving PatientId), **publishes** `prescription.created` , audits. Active-list read via stored proc.

### MedicalRecordService

Saves the uploaded blob via `IFileStorage` , persists the record as **Pending**, then **publishes** `medical-record.uploaded` — the request returns immediately while a worker summarizes asynchronously. File download audits `MedicalRecordViewed` and streams the original blob back.

### DashboardService

**Cache-aside** over Redis (key `dashboard:stats` , TTL 60s). On miss: KPI scalars from the stored proc + a 7-day EF group-by (zero-filled) for the trend chart, then cached.

### AiAssistantService

Doctor-facing AI. Symptom analysis and a **grounded (RAG) patient-history chat**: it builds a context block from the patient's real demographics, last 20 appointments, 30 prescriptions, and 10 record summaries, then asks the LLM to answer using *only* that context. Audits every AI call.

### PatientPortalService

The security heart of the portal. Every method funnels through `MeAsync()` , which resolves the caller's `UserId` → their Patient record. If there's no linked profile, or a requested record/prescription isn't theirs, it throws `403` . Patients can only ever touch their own data.

### ClinicPortalService

Staff side of the portal: provision a patient login (with conflict checks → `409` , BCrypt-hashed password, links Patient↔User), triage refills (approve/deny with resolver + timestamp), and reply to message threads. Audits account creation & refill resolution.

**Consistent error model:** all services throw `AppException` carrying an HTTP status ( `NotFound` →404, `Forbidden` →403, `Conflict` →409, default 400). A single middleware translates these into clean JSON responses — controllers stay thin.

## Validation

FluentValidation validators run in a global `ValidationFilter` before controller code. Examples: login email/password required; new-patient DOB must be in the past; appointment `ScheduledAt` must be in the future; portal passwords ≥ 8 chars; symptom text 3–4000 chars; message body ≤ 4000. Invalid input never reaches business logic.



## 7 · Event-Driven Architecture (Kafka)

Slow, best-effort, or fan-out work is decoupled from the request via Kafka. The API produces events; workers consume them.

| Topic                     | Producer → Consumer               | What happens                                            |
|---------------------------|-----------------------------------|---------------------------------------------------------|
| medical-record.uploaded   | API → AI Summary Worker           | Worker extracts text, calls the LLM, writes the summary |
| medical-record.summarized | AI Worker → API realtime consumer | API pushes "SummaryReady" to the browser over SignalR   |
| appointment.booked        | API → Notification Worker         | Sends reminder (logged in MVP; Twilio/SendGrid in prod) |
| audit.event               | API → Audit Worker                | Persists an immutable audit-log row                     |
| prescription.created      | API → (audit)                     | Downstream audit / future hooks                         |
| <topic>.dlq               | Any consumer → dead-letter        | Poison messages after retries land here for inspection  |

### Producer guarantees

- Single **singleton** Kafka producer, configured `Acks=All` + `EnableIdempotence=true` — no duplicate/ lost writes on retry.
- Graceful shutdown flushes pending messages (5s drain).
- Messages are JSON with a partition key (usually the entity id) so per-entity ordering is preserved.

### Consumer guarantees (shared `KafkaConsumerBase` )

- **At-least-once**: auto-commit is **off**; the offset is committed only after successful handling.
- Each message is handled in a **fresh DI scope** (so a scoped DbContext per message).
- **Retry with backoff**: up to 3 attempts, linear `500ms × attempt` .
- **Dead-letter**: after exhausting retries, the message is republished to `<topic>.dlq` with the original payload + error, then the offset is committed so one poison message can't block the partition.
- Each worker uses a **distinct consumer group**, so they scale and partition independently.

**Honest nuance to raise proactively:** at-least-once means handlers should be idempotent. The audit worker could in theory insert a duplicate row if a message were reprocessed after a mid-batch crash — a known trade-off of choosing at-least-once over exactly-once.

### The three workers

| Worker             | Consumes                | Does                                                                                                                                                          |
|--------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AISummaryWorker    | medical-record.uploaded | Sets Processing → extracts text (PdfPig) → LLM summarize → sets Ready + AISummary → publishes summarized . On failure sets Failed and re-throws (→ retry/DLQ) |
| NotificationWorker | appointment.booked      | Loads appointment projection, logs a reminder (prod: SMS/email)                                                                                               |
| AuditWorker        | audit.event             | Maps the event to an AuditLog row and persists it immutably                                                                                                   |

## 8 · Real-Time Layer (SignalR)

How does the browser know an AI summary finished without polling forever? A hosted consumer inside the API bridges Kafka → SignalR.

```
Doctor uploads record
  | POST /api/medical-records      (returns 202-ish immediately, status = Pending)
  v
API publishes medical-record.uploaded → AiSummaryWorker (LLM summarize)
                                         | publishes medical-record.summarized
                                         v
                                RecordSummarizedConsumer (hosted in API)
                                         | IHubContext.Clients.Group("patient:{id}")
                                         v
SignalR —"SummaryReady"—> Browser
                                         |
React Query invalidates ['records', id] → UI refetches → "Ready"
```

- The `NotificationsHub` is `[Authorize]`d; SignalR clients pass the JWT via the `access_token` query string (handled in the JWT bearer `OnMessageReceived` event).
- Clients join a **per-patient group** ( `SubscribeToPatient` ) so pushes are targeted, not broadcast.
- A **Redis backplane** fans messages out across API replicas — the consumer that received the Kafka event may not be the replica holding the client's WebSocket.
- The frontend keeps a **3-second polling backstop** so readiness is never missed if the socket drops.

## 9 · Security & Authentication

### Passwords

**BCrypt** with a per-hash salt ( `BCryptPasswordHasher` ). Never stored or logged in plaintext.

### Access tokens

Stateless **JWT**, HMAC-SHA256 signed. Claims: `sub` (user id), `jti` , name, role. Issuer/audience/lifetime/signature all validated. 60-minute lifetime.

### Refresh tokens

32 cryptographically-random bytes, stored in Redis under `refresh:{token}` with a 7-day TTL. **Single-use**: consuming one deletes the key and issues a new pair (rotation) — replay-resistant.

### RBAC

Three roles: Doctor, Receptionist, Patient. Enforced by `[Authorize(Roles=...)]` . "Staff" = Doctor+Receptionist. Patient routes live under `/api/portal` and are Patient-only.

### Defense in depth

- **Endpoint-level RBAC** (staff endpoints reject patients with 403) **plus data-level scoping** (the portal's `MeAsync` guarantees a patient can only read their own records — belt and braces).
- **Immutable audit trail**: sensitive access/mutations (record views, patient reads, diagnoses, account creation, refill resolution) are published to Kafka and persisted by the audit worker — tamper-resistant because it's append-only and written out-of-band.
- **Path-traversal guards** on file storage ( `Path.GetFileName` on save & read).
- **Generic auth errors** to avoid user enumeration.
- **CORS** restricted to configured origins with credentials; **rate limiting** on AI endpoints.
- SignalR hub requires authentication; per-patient groups prevent cross-patient leakage.

## 10 · AI Subsystem

All AI sits behind `IAIService` with two implementations selected by config ( `Ai:Provider` ): a deterministic **stub** (default, no API key) and a real **OpenAI** adapter.

| Feature                    | How it works                                                                                                                                                                                    |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Symptom analysis           | LLM returns <b>strict JSON</b> — possible conditions, urgency (Low/Med/High), suggested tests, disclaimer. Stub uses keyword heuristics (e.g. "chest pain" → High).                             |
| Record summarization       | Async worker extracts document text and asks the LLM for a structured summary (overview, key findings, meds, allergies, recommendations).                                                       |
| Patient-history chat (RAG) | Context is built <b>only</b> from the patient's real DB records; the LLM is instructed to answer solely from that context — grounding reduces hallucination and keeps answers patient-specific. |

- Real provider is a **typed HttpClient wrapped with Polly** (retries, circuit breaker, timeout) — the costly external dependency can't hang the app.
- AI endpoints are **rate-limited** (10/min) and **audited**.
- The stub means the whole system runs end-to-end offline in CI/demo with zero secrets.

## 11 · Cross-Cutting Concerns

| Concern                | Implementation                                                                               |
|------------------------|----------------------------------------------------------------------------------------------|
| Caching                | Redis cache-aside on the dashboard (60s TTL); <code>AsNoTracking</code> on reads             |
| Resilience             | Polly standard handler on the AI client; Kafka retry + DLQ; idempotent producer              |
| Health checks          | Deep <code>/health</code> probes the real Postgres, Redis, and Kafka — not just "process up" |
| Observability          | OpenTelemetry tracing + metrics for ASP.NET & HttpClient; OTLP export or console in dev      |
| Rate limiting          | Fixed-window limiter (10/min) on AI endpoints → 429 when exceeded                            |
| Error handling         | <code>ExceptionHandlerMiddleware</code> maps exceptions → consistent JSON + status           |
| API docs               | Swagger UI with Bearer auth wired in (dev)                                                   |
| Config-driven adapters | AI provider, storage, migrate-on-startup all switch via configuration                        |

## 12 · Frontend Architecture (Next.js 16)

### Structure

- **Route groups** split the two authenticated shells: `(staff)/*` (clinic app) and `portal/*` (patient portal), plus shared `/` landing & `/login`.
- Each group has a **client layout that guards access**: waits for auth hydration, redirects unauthenticated users to `/login`, and redirects cross-role (a patient hitting staff → portal, and vice-versa).
- Finer gating inside pages: doctor-only controls (upload, prescribe, diagnose) via `isDoctor`, receptionist-only account provisioning via `isReceptionist`.

### State & data

- **Zustand** holds *only* auth (tokens, name, role) — persisted to `localStorage`, SSR-safe (no-op storage on the server), with a `hydrated` flag so redirects don't fire before the token loads.
- **React Query** is the entire server-state layer: structured query keys, `enabled` gating on ids, mutation-driven invalidation, and **dynamic polling** — 3s while an AI summary is Pending/Processing, 5s for message threads.
- **Axios instance** with interceptors: attaches the Bearer token per request; on `401` runs a **single-flight refresh** (one refresh call shared across concurrent failures), replays the original request, or logs out if refresh fails.

### Realtime & theming

- **SignalR singleton** connects to `/hubs/notifications` with the JWT via `accessTokenFactory` and auto-reconnect; `subscribeToPatient` wires "SummaryReady" → invalidate the records query. Errors are swallowed (the 3s poll is the backstop).
- **MUI theme**: `createTheme({ cssVariables:true })`, teal/indigo healthcare palette, flat bordered cards, Geist typography — provided via `AppRouterCacheProvider` → `ThemeProvider` → `CssBaseline` → `QueryClientProvider`.
- **Dashboard** uses MUI X Charts for the 7-day trend; a shared `<Thread>` component renders chat bubbles for both patient and staff sides by comparing `fromPatient` to the viewer.

## 13 · End-to-End Feature Walkthroughs

---

These tie the layers together — great for "walk me through a feature" questions.

### A · Uploading a medical record (async AI + realtime)

1. Doctor uploads a PDF → `POST /api/medical-records` (multipart).
2. `MedicalRecordService` validates the patient, saves the blob ( `IFileStorage` ), persists the record as **Pending**, publishes `medical-record.uploaded`, audits, returns immediately.
3. UI shows "Processing" and polls every 3s (React Query backstop).
4. `AISummaryWorker` consumes the event → Processing → extracts text (PdfPig) → LLM summary → Ready + `AISummary` → publishes `medical-record.summarized`.
5. `RecordSummarizedConsumer` (in API) pushes "SummaryReady" to the patient's SignalR group.
6. Browser invalidates `['records', id]`, refetches, shows the summary. Status flips to **Ready**.

### B · Patient self-books an appointment (data-scoped security)

1. Patient calls `POST /api/portal/appointments`.
2. `PatientPortalService.BookAsync` → `MeAsync()` resolves the caller's own PatientId from the JWT (a patient can't book for someone else).
3. Delegates to `AppointmentService.CreateAsync` → validates doctor, creates appointment, publishes `appointment.booked`, audits.
4. `NotificationWorker` consumes and sends the reminder.

### C · Login + token refresh (security lifecycle)

1. `POST /api/auth/login` → BCrypt verify → JWT access token + random refresh token (Redis, 7-day TTL).
2. Frontend stores both in Zustand/localStorage; axios attaches the access token on every request.
3. On a `401`, the axios interceptor calls `/api/auth/refresh` once (single-flight); the server **consumes & rotates** the refresh token and issues a new pair; the original request replays.
4. If refresh fails, the client logs out.

## 14 · Interview Q&A — Likely Questions

---

### Q. Why Clean Architecture here — isn't it overkill for an MVP?

It buys testability and swappable adapters at almost no cost. Concretely: the AI provider, file storage, and messaging are all behind interfaces, so the app runs offline with a stub AI and local-disk storage, and moving to OpenAI or S3 is a one-line config/registration change — no business logic touched. Services are unit-tested without a database or Kafka.

### Q. Why Kafka instead of just doing the work in the request?

AI summarization can take seconds and calls an external API; audit writes and notifications are best-effort. Putting them inline would make uploads slow and couple the request to flaky externals. Kafka lets the API return immediately, gives durable at-least-once delivery, retries + dead-lettering, and lets workers scale independently.

### Q. How do you guarantee delivery / handle failures in the pipeline?

Producer is idempotent with `Acks=ALL`. Consumers disable auto-commit and commit only after success (at-least-once), retry 3× with backoff, and dead-letter poison messages to `<topic>.dlq` before committing so one bad message can't block a partition. The trade-off is handlers should be idempotent, which I'd harden next.

### Q. How does the UI update when async work finishes?

A hosted consumer in the API listens for the "summarized" event and pushes a SignalR message to the patient's group; the client invalidates the relevant React Query and refetches. A Redis backplane makes that work across API replicas, and a 3-second poll is a backstop if the socket drops.

### Q. How is a patient prevented from seeing another patient's data?

Two layers. RBAC blocks patients from staff endpoints (403). And every portal method funnels through `MeAsync()`, which resolves the caller's own PatientId from the JWT and checks ownership on records/prescriptions — so even a crafted id can't reach someone else's data.

### Q. Why stored procedures with an EF Core app?

For the heavy aggregate reads — dashboard KPIs, paginated search with a total count, history — DB-side SQL with window functions is one round-trip and avoids materializing rows in memory. Writes and simple reads stay in EF for productivity. The raw SQL is isolated behind a helper so it doesn't leak into the domain.

### Q. How would you take this to production / add HIPAA?

The scaffolding is intentionally there: an immutable audit trail, RBAC, and a storage abstraction. I'd add encryption at rest + a real object store (S3/Blob) behind the existing `IFileStorage`, TLS everywhere, secrets in a vault, tighten to exactly-once-effect via idempotent handlers, add integration tests, and put the audit log behind a write-once store. None of that changes the core architecture.

### Q. How do you ground the AI so it doesn't hallucinate patient facts?

The chat is a lightweight RAG: I assemble context strictly from the patient's real records (demographics, recent appointments, prescriptions, record summaries) and instruct the model to answer using only that context. Symptom analysis and summaries use strict-JSON prompts so responses are parseable and bounded, and every AI call is rate-limited and audited.

### Q. What are the weak spots you'd improve?

Idempotent consumers to fully tame at-least-once duplicates; broader test coverage (integration + E2E, not just unit); real notification providers; moving migrate-on-startup to a controlled migration step for prod; and adding authorization policies as named policies rather than role strings for finer control.

### Q. Walk me through the single most impressive flow.

The record-upload pipeline (Walkthrough A): one user action crosses HTTP, blob storage, Kafka, a background worker, an external LLM, a second Kafka topic, an in-API consumer, a Redis-backed SignalR push, and finally a React Query invalidation that updates the UI live — while the original request returned in milliseconds. It exercises essentially every part of the system.

**Closing line:** "MediTrack is small in scope but deliberately production-shaped — the interesting parts aren't the CRUD, they're the seams: the event backbone, the async-to-realtime bridge, the two-layer security model, and the swappable adapters that let it run offline yet scale to real infrastructure without rewrites."